

Отчет по практической работе №4

Выполнил: Элякова Алеся Львовна

Группа: БСБО-16-23

Дата выполнения: 19.05.2026

1. Настройка хранилища

1.1 Выбранный вариант хранилища

Вариант A: Local Path Provisioner

1.2 StorageClass

```
root@k8s-master:~# kubectl get storageclass
NAME                PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE   ALLOWVOLUMEEXPANSION   AGE
local-path (default) rancher.io/local-path   Delete          WaitForFirstConsumer false                 11d
root@k8s-master:~#
```

1.3 Тестовый PVC

```
root@k8s-master:~# kubectl get pvc
NAME                STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTES
test-pvc            Bound     pvc-fc4c356d-8b1d-410b-82db-118410f73c83  1Gi        RWO             local-path     <unset>
root@k8s-master:~# kubectl get pv
NAME                CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS   CLAIM                S
TORAGECLASS        VOLUMEATTRIBUTESCLASS REASON          AGE
pvc-dfc6873e-da25-4cf7-bdc5-28b52fbc1045  5Gi        RWO             Delete           Bound    lab3-app/postgres-pvc  l
ocal-path          <unset>      11d
pvc-fc4c356d-8b1d-410b-82db-118410f73c83  1Gi        RWO             Delete           Bound    default/test-pvc       l
ocal-path          <unset>      48s
root@k8s-master:~# kubectl get pods
NAME    READY   STATUS    RESTARTS   AGE
test-pod 1/1     Running   0          63s
```

2. Установка Helm

2.1 Версия Helm

```
root@k8s-master:~# wget https://get.helm.sh/helm-v3.13.2-linux-amd64.tar.gz
--2026-05-19 15:26:27-- https://get.helm.sh/helm-v3.13.2-linux-amd64.tar.gz
Resolving get.helm.sh (get.helm.sh)... 150.171.109.101, 2603:1061:14:64::1
Connecting to get.helm.sh (get.helm.sh)|150.171.109.101|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 16185334 (15M) [application/x-tar]
Saving to: 'helm-v3.13.2-linux-amd64.tar.gz'

helm-v3.13.2-linux-amd64.tar. 100%[=====] 15.43M 8.52MB/s in 1.8s

2026-05-19 15:26:35 (8.52 MB/s) - 'helm-v3.13.2-linux-amd64.tar.gz' saved [16185334/16185334]

root@k8s-master:~# tar -zxvf helm-v3.13.2-linux-amd64.tar.gz
linux-amd64/
linux-amd64/helm
linux-amd64/LICENSE
linux-amd64/README.md
root@k8s-master:~# sudo mv linux-amd64/helm /usr/local/bin/helm
root@k8s-master:~# helm version
version.BuildInfo{Version:"v3.13.2", GitCommit:"2a2fb3b98829f1e0be6fb18af2f6599e0f4e8243", GitTreeState:"clean", GoVersion:"go1.20.10"}
```

2.2 Репозитории

```
root@k8s-master:~# helm repo list
NAME      URL
bitnami   https://charts.bitnami.com/bitnami
stable    https://charts.helm.sh/stable
root@k8s-master:~#
```

3. Создание собственного Helm-чарта

3.1 Структура чарта

```
root@k8s-master:~/helm-charts# tree lab4-app/
lab4-app/
├── charts
├── Chart.yaml
├── templates
│   ├── deployment.yaml
│   ├── _helpers.tpl
│   ├── hpa.yaml
│   ├── ingress.yaml
│   ├── NOTES.txt
│   ├── serviceaccount.yaml
│   ├── service.yaml
│   └── tests
│       └── test-connection.yaml
└── values.yaml

4 directories, 10 files
```

3.2 Установка чарта

```
root@k8s-master:~/helm-charts# helm list -n lab4-app
```

NAME	NAMESPACE	REVISION	UPDATED	STATUS	CHART	APP VERSION
lab4-release	lab4-app	2	2026-05-20 17:06:44.435013163 +0300 MSK	deployed	lab4-app-0.1.0	1.0.0

3.3 Сгенерированные ресурсы

```

root@k8s-master:~/helm-charts# kubectl get all -n lab4-app
NAME                                READY    STATUS    RESTARTS   AGE
pod/lab4-release-goapp-7cf4d8b4d-2m6js    1/1      Running   0           3m47s
pod/lab4-release-goapp-7cf4d8b4d-nqc2x    1/1      Running   0           3m44s
pod/lab4-release-goapp-7cf4d8b4d-t8ncd    1/1      Running   0           3m41s
pod/lab4-release-nginx-756f98b598-cdkxn    1/1      Running   0           16m
pod/lab4-release-nginx-756f98b598-sm8pf    1/1      Running   0           16m
pod/lab4-release-postgres-5446f5897-cwlvl    1/1      Running   0           16m

NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/lab4-release-goapp          ClusterIP           10.97.172.64    <none>            8080/TCP         20m
service/lab4-release-nginx          ClusterIP           10.96.230.126   <none>            80/TCP           20m
service/lab4-release-postgres       ClusterIP           10.111.251.92   <none>            5432/TCP         20m

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/lab4-release-goapp    3/3      3              3            20m
deployment.apps/lab4-release-nginx    2/2      2              2            20m
deployment.apps/lab4-release-postgres 1/1      1              1            20m

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/lab4-release-goapp-67bd8d6d5b    0          0          0        15m
replicaset.apps/lab4-release-goapp-76ddd59f56    0          0          0        6m41s
replicaset.apps/lab4-release-goapp-7cf4d8b4d      3          3          3        3m47s
replicaset.apps/lab4-release-nginx-756f98b598     2          2          2        20m
replicaset.apps/lab4-release-postgres-5446f5897   1          1          1        20m
root@k8s-master:~/helm-charts# kubectl get pvc -n lab4-app
NAME                                STATUS    VOLUME                                     CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESC
lab4-release-postgres-pvc          Bound    pvc-98a4cf68-0cee-4847-aeaa-7dcf3849e147   5Gi        RWO              local-path      <unset>
root@k8s-master:~/helm-charts#

```

4. PersistentVolumes

4.1 PV и PVC

```

root@k8s-master:~/helm-charts# kubectl get pv,pvc --all-namespaces
NAME                                STORAGECLASS    VOLUMEATTRIBUTESCLASS    REASON    CAPACITY   ACCESS MODES   RECLAIM POLICY    STATUS    CLAIM
persistentvolume/pvc-a44c93b9-aea0-44bc-b22c-4c46345f1536    local-path      <unset>                                5Gi        RWO              Delete             Bound     lab4-app/lab4-release-postgres-pvc
persistentvolume/pvc-dfc6873e-da25-4cf7-bdc5-28b52fbc1045    local-path      <unset>                                5Gi        RWO              Delete             Bound     lab3-app/postgres-pvc
persistentvolume/pvc-fc4c356d-8b1d-410b-82db-118410f73c83    local-path      <unset>                                1Gi        RWO              Delete             Bound     default/test-pvc

NAMESPACE    NAME                                STATUS    VOLUME                                     CAPACITY   ACCESS MODES
default      persistentvolumeclaim/test-pvc      Bound     pvc-fc4c356d-8b1d-410b-82db-118410f73c83    1Gi        RWO
lab3-app     persistentvolumeclaim/postgres-pvc  Bound     pvc-dfc6873e-da25-4cf7-bdc5-28b52fbc1045    5Gi        RWO
lab4-app     persistentvolumeclaim/lab4-release-postgres-pvc  Bound     pvc-a44c93b9-aea0-44bc-b22c-4c46345f1536    5Gi        RWO
root@k8s-master:~/helm-charts#

```

4.2 Тест сохранности данных

API в Go приложении не работает, поэтому я добавил нового пользователя через psql:

```

root@k8s-master:~/helm-charts# kubectl exec -n lab4-app lab4-release-postgres-c47995887-v965l -- psql -U postgres -d myapp -c "INSERT INTO users (name) VALUES ('PV-Test-User');"
INSERT 0 1
root@k8s-master:~/helm-charts# kubectl exec -n lab4-app lab4-release-goapp-74bf46d999-7j646 -- wget -q -O- http://localhost:8080/api/users >/dev/null
[{"id":4,"name":"PV-Test-User"}, {"id":3,"name":"DevOps Pro"}, {"id":2,"name":"Chart Master"}, {"id":1,"name":"HelM User"}]
root@k8s-master:~/helm-charts#

root@k8s-master:~/helm-charts# kubectl delete pod -n lab4-app -l app.kubernetes.io/component=postgresql
pod "lab4-release-postgres-c47995887-v965l" deleted
root@k8s-master:~/helm-charts# kubectl get pods -n lab4-app -w
NAME                                READY    STATUS    RESTARTS   AGE
lab4-release-goapp-74bf46d999-7j646    1/1      Running   0           16m
lab4-release-goapp-74bf46d999-bs5wm    1/1      Running   0           16m
lab4-release-goapp-74bf46d999-gxx5f    1/1      Running   0           16m
lab4-release-goapp-74bf46d999-pjgwb    1/1      Running   0           16m
lab4-release-goapp-74bf46d999-wl4bt    1/1      Running   0           16m
lab4-release-nginx-756f98b598-24txq    1/1      Running   0           34m
lab4-release-nginx-756f98b598-dh55p    1/1      Running   0           24m
lab4-release-nginx-756f98b598-m486z    1/1      Running   0           34m
lab4-release-postgres-c47995887-5cjkg    1/1      Running   0           9s
^Croot@k8s-master:~/helm-charts# kubectl exec -n lab4-app $(kubectl get pod -n lab4-app -l app.kubernetes.io/component=postgresql -o name | head -1) -- psql -U postgres -d myapp -c "SELECT * FROM users;"
id | name | created_at
---+---+-----
 1 | HelM User | 2026-05-20 14:59:19.478758
 2 | Chart Master | 2026-05-20 14:59:19.478758
 3 | DevOps Pro | 2026-05-20 14:59:19.478758
 4 | PV-Test-User | 2026-05-20 15:31:55.773959
(4 rows)
root@k8s-master:~/helm-charts#

```

4.3 Тест миграции (если выполнялся)

```
eal@k8s-worker2:~$ su - root
Password:
root@k8s-worker2:~# sudo systemctl stop kubelet
root@k8s-worker2:~#
```

```
root@k8s-master:~/helm-charts# kubectl get pods -n lab4-app -w
NAME                                READY    STATUS              RESTARTS    AGE
lab4-release-goapp-74bf46d999-7j646 1/1      Running             0            37m
lab4-release-goapp-74bf46d999-bs5wm 1/1      Running             0            37m
lab4-release-goapp-74bf46d999-gxx5f 1/1      Running             0            37m
lab4-release-goapp-74bf46d999-pjgwb 1/1      Running             0            37m
lab4-release-goapp-74bf46d999-wl4bt 1/1      Running             0            37m
lab4-release-nginx-fd4948fd9-p4bmz 1/1      Running             0            7m24s
lab4-release-nginx-fd4948fd9-sjcbj 1/1      Running             0            7m22s
lab4-release-nginx-fd4948fd9-wdc72 1/1      Running             0            7m21s
lab4-release-postgres-c47995887-5cjkg 1/1      Running             0            20m
test-curl                            0/1      CrashLoopBackOff    7 (70s ago)  12m
lab4-release-postgres-c47995887-5cjkg 1/1      Running             0            20m
lab4-release-nginx-fd4948fd9-wdc72 1/1      Running             0            7m43s
lab4-release-goapp-74bf46d999-bs5wm 1/1      Running             0            37m
lab4-release-goapp-74bf46d999-pjgwb 1/1      Running             0            37m
```

5. Управление релизами

5.1 История обновлений

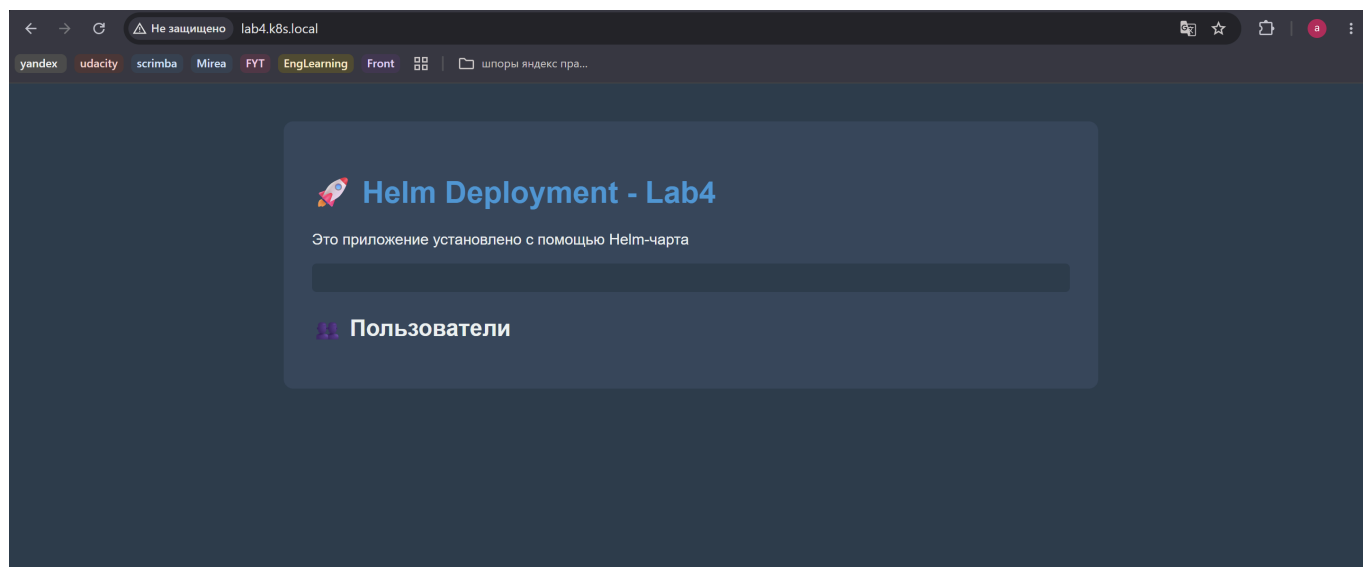
```
root@k8s-master:~/helm-charts# helm history lab4-release -n lab4-app
REVISION    UPDATED                     STATUS      CHART          APP VERSION    DESCRIPTION
1           Wed May 20 17:59:11 2026    superseded  lab4-app-0.1.0 1.0.0          Install complete
2           Wed May 20 18:03:36 2026    superseded  lab4-app-0.1.0 1.0.0          Upgrade complete
3           Wed May 20 18:05:13 2026    failed      lab4-app-0.1.0 1.0.0          Upgrade "lab4-release" failed: cannot patch "lab4-release-postgres-pvc" with kind PersistentVolumeClaim: persistentvolumeclaims "lab4-release-postgres-pvc" is forbidden: only dynamically provisioned pvc can be resized and the storageclass that provisions the pvc must support resize
4           Wed May 20 18:05:58 2026    superseded  lab4-app-0.1.0 1.0.0          Upgrade complete
5           Wed May 20 18:07:51 2026    superseded  lab4-app-0.1.0 1.0.0          Rollback to 1
6           Wed May 20 18:08:55 2026    superseded  lab4-app-0.1.0 1.0.0          Rollback to 4
7           Wed May 20 18:10:46 2026    deployed   lab4-app-0.1.0 1.0.0          Upgrade complete
```

5.2 Результат отката

```
root@k8s-master:~/helm-charts# kubectl get pods -n lab4-app
NAME                                READY    STATUS              RESTARTS    AGE
lab4-release-goapp-74bf46d999-7j646 1/1      Running             0            47m
lab4-release-goapp-74bf46d999-bs5wm 1/1      Running             0            47m
lab4-release-goapp-74bf46d999-gxx5f 1/1      Running             0            47m
lab4-release-goapp-74bf46d999-pjgwb 1/1      Running             0            47m
lab4-release-goapp-74bf46d999-wl4bt 1/1      Running             0            47m
lab4-release-nginx-fd4948fd9-p4bmz 1/1      Running             0            17m
lab4-release-nginx-fd4948fd9-sjcbj 1/1      Running             0            17m
lab4-release-nginx-fd4948fd9-wdc72 1/1      Running             0            17m
lab4-release-postgres-c47995887-5cjkg 1/1      Running             0            30m
root@k8s-master:~/helm-charts#
```

7. Скриншоты

7.1 Главная страница приложения



7.2 Дашборд с PVC информацией

Дашборд не работает из-за проблем с Go кодом. Однако Helm кластер может отобразить информацию:

```
root@k8s-master:~/helm-charts# kubectl describe pvc -n lab4-app
Name:          lab4-release-postgres-pvc
Namespace:     lab4-app
StorageClass:  local-path
Status:        Bound
Volume:        pvc-a44c93b9-aea0-44bc-b22c-4c46345f1536
Labels:        app.kubernetes.io/component=postgresql
               app.kubernetes.io/managed-by=Helm
               app.kubernetes.io/version=1.0.0
               helm.sh/chart=lab4-app-0.1.0
Annotations:   meta.helm.sh/release-name: lab4-release
               meta.helm.sh/release-namespace: lab4-app
               pv.kubernetes.io/bind-completed: yes
               pv.kubernetes.io/bound-by-controller: yes
               volume.beta.kubernetes.io/storage-provisioner: rancher.io/local-path
               volume.kubernetes.io/selected-node: k8s-worker2
               volume.kubernetes.io/storage-provisioner: rancher.io/local-path
Finalizers:    [kubernetes.io/pvc-protection]
Capacity:      5Gi
Access Modes:  RWO
VolumeMode:    Filesystem
Used By:       lab4-release-postgres-c47995887-5cjkg
Events:        <none>
root@k8s-master:~/helm-charts#
```

7.3 Данные в PostgreSQL после миграции

Также пользователи не отображаются из-за проблем в Go коде. Однако данные о пользователях хранятся в postgres подах:

```
root@k8s-master:~/helm-charts# kubectl exec -n lab4-app $(kubectl get pod -n lab4-app -l app.kubernetes.io/component=postgresql -o name | head -1) -- psql -U postgres -d myapp -c "SELECT * FROM users;"
 id |      name      |      created_at
----+-----+-----
  1 | Helm User      | 2026-05-20 14:59:19.478758
  2 | Chart Master   | 2026-05-20 14:59:19.478758
  3 | DevOps Pro     | 2026-05-20 14:59:19.478758
  4 | PV-Test-User   | 2026-05-20 15:31:55.773959
(4 rows)
```

8. Ответы на контрольные вопросы

1. В чем разница между emptyDir, hostPath и PersistentVolume?

emptyDir - временный том, создаётся вместе с подом и уничтожается вместе с ним. Данные живут только пока под запущен, используется для временных файлов или обмена данными между контейнерами внутри пода.

hostPath - монтирует директорию прямо с файловой системы ноды. Данные сохраняются после перезапуска пода, но привязаны к конкретной ноде - если под переедет на другую ноду, данные будут недоступны.

PersistentVolume (PV) - независимый ресурс кластера, не привязан ни к поду ни к ноде. Данные сохраняются при перезапуске и миграции пода, управляется через PVC.

2. Как работает динамическое предоставление томов (dynamic provisioning)?

Без динамического provisioning администратор должен вручную создавать PV заранее. При динамическом provisioning всё происходит автоматически: пользователь создаёт PVC с указанием StorageClass, Kubernetes обращается к provisioner, provisioner автоматически создаёт PV и связывает его с PVC.

3. Что такое StorageClass и зачем он нужен?

StorageClass - это шаблон для создания томов, который описывает какой provisioner использовать, тип хранилища и его параметры. Позволяет иметь несколько классов хранилищ и выбирать нужный при создании PVC. В данной практике использовался `local-path`.

4. Какие преимущества дает Helm по сравнению с набором YAML-файлов?

- Одни и те же манифесты переиспользуются с разными значениями через `values.yaml`.
 - Helm отслеживает историю установок, можно откатиться на предыдущую версию через `helm rollback`
 - Можно подключать сторонние чарты (например, официальный PostgreSQL) через `Chart.yaml`
 - Одна команда `helm install` вместо множества `kubectl apply`
-

5. Как Helm управляет зависимостями между компонентами?

Helm управляет зависимостями через файл `Chart.yaml` в секции `dependencies`, где указываются сторонние чарты. Они скачиваются командой `helm dependency update` и сохраняются в папку `charts/`. Внутри одного чарта порядок создания ресурсов управляется через хуки (`helm hooks`).

6. Что происходит с данными при миграции пода на другой узел в разных типах хранилищ?

emptyDir - данные теряются полностью, том создаётся заново на новой ноде

hostPath - данные остаются на старой ноде и недоступны на новой

PersistentVolume - зависит от типа: сетевые хранилища доступны с любой ноды и данные сохраняются; локальные PV привязаны к ноде и под будет перезапланирован на ту же ноду

9. Выводы

В ходе работы я создала Helm-чарт, разобралась в работе PV и PVC сущностей в Kubernetes. В процессе возникали трудности с синтаксисом YAML файлов, которые возникали из-за отсутствия опыта работы с шаблонами в YAML файле. Helm кардинально меняет подход к управлению приложениями: вместо набора несвязанных YAML-файлов появляется единый версионированный артефакт с возможностью отката, параметризации под разные окружения и управления зависимостями.